

音楽の数学 探求ツール — 詳細技術仕様書 (計画書の別冊)

music-math-explorer-plan.md の実装詳細版。アルゴリズム・数式・手順・リスクまで踏み込む

Part A. コア技術の詳細

A-1. 音の生成(Tone.js / Web Audio API)

必要な知識と実装ポイント

- 平均律の周波数計算: $f(n) = 440 \times 2^{((n-69)/12)}$ (nはMIDIノート番号、A4=440Hz=69)
- 純正律の周波数計算: 基準音に対する比率で直接計算(五度= $\times 3/2$ 、長三度= $\times 5/4$ 、完全四度= $\times 4/3$)
- うなり(beat)を聞かせるための実装: 2つのOscillatorを同時に鳴らす。平均律の五度($2^{(7/12)} \approx 1.4983$)と純正の五度(1.5)では、上音同士が微妙にズれて毎秒数回のうなりが生じる。正弦波(sine)だと基音同士のうなりしか聞こえないため、**倍音を含む波形(sawtooth/triangle)またはカスタム倍音(Tone.jsのPartialsで倍音列を明示指定)を使うこと**
- AudioContextはユーザー操作(クリック)後でないとは起動できないブラウザ制約がある → 「音を有効化」ボタンを共通コンポーネントに必ず入れる
- 音量はdB管理(Tone.jsのvolume)し、複数音重ねる際のクリッピングを防ぐため -12dB程度をデフォルトに

落とし穴

- iOS SafariはWeb Audioの自動再生制限が厳しい。必ずタッチイベント内でTone.start()を呼ぶ
- Oscillatorは使い捨て(1回stopしたら再利用不可)。再生ごとに生成する設計にする

Part A-2. 音声入力アーキテクチャ(統合設計)

設計思想: ユーザーは「マイク」か「ファイル」を入れるだけ。内部で自動判定し、どちらの経路でも最終的に同じ内部フォーマット(ピッチクラス・音符・和音のタイムスタンプ付きリスト)に正規化してから、10個の数学モジュールに渡す。モジュール側は入力方式を一切意識しない。

入力(マイク or ファイル)

↓

[入力振り分け]

└─ マイクストリーム

└─ → YIN (A-2-1) でリアルタイム処理、単音専用、遅延<100ms目標

└─

└─ 音声ファイル (クラシック曲等、和音・複数声部を含む)

└─ → タイムストレッチ (A-2-3) で時間分解能を確保 (任意、速い曲のみ)

└─ → Basic Pitch / TensorFlow.js版 (A-2-4) で和音を含めて解析

└─ → 処理に数秒〜曲の長さに応じた時間がかかる (非リアルタイム)

↓

[正規化アダプター] どちらの経路も

{ time, pitchClass, midiNote, confidence } の共通配列に変換

↓

数学モジュール (Tonnetz・フーリエ解析・群論ブレイグラウンド等)

※ここから先はA-2-1/A-2-4のどちらが生成したデータか区別しない

A-2-1. リアルタイム経路(YIN)

アルゴリズムの中身(理解して実装すべきこと)

YINは自己相関ベースのピッチ検出で、以下のステップからなる:

1. 差分関数: $d(\tau) = \sum (x[j] - x[j+\tau])^2$ — 信号を τ サンプルずらした自分自身との差
2. 累積平均正規化: $d'(\tau) = d(\tau) / [(1/\tau) \sum d(j)]$ — 低周波側の誤検出を防ぐ正規化
3. 閾値(通常0.1~0.15)を最初に下回る τ を探す
4. 放物線補間で τ をサブサンプル精度に補正 → $f = \text{sampleRate} / \tau$

実装手順

1. `navigator.mediaDevices.getUserMedia({audio: true})` でマイクストリーム取得
2. `AudioContext.createAnalyser()` または `AudioWorklet` でPCMデータを取得(バッファサイズ2048推奨: 小さいと低音が検出できず、大きいとレイテンシ増)
3. Pitchfinder.jsのYIN実装に流す(自作しなくてよいが、パラメータの意味は理解しておく)
4. 検出周波数 → MIDIノート番号 → ピッチクラス(mod 12)に変換
5. メディアンフィルタ(直近5フレームの中央値)を必ずかける。生の検出値はオクターブエラーや瞬間的な誤検出でジャンプするため

性能目標と検証方法

- 目標: 検出遅延 < 100ms(バッファ2048 @ 44.1kHz ≈ 46ms + 処理時間)
- Day1にPoC(マイク→周波数表示だけの最小ページ)を作り、これを実測してから本実装に進む。
ここが想定より重い場合、リアルタイム層の設計全体を見直す必要があるため、最初に潰すべき

最大のリスク

落とし穴

- 倍音の強い楽器(ギター等)ではオクターブ下/上への誤検出が頻発 → メディアンフィルタ+閾値調整で緩和、完全には消えない旨をUIで正直に表示
- 環境ノイズ対策: RMSレベルが閾値未満のフレームは「無音」として棄却
- HTTPSでないとgetUserMediaが動かない(localhostは例外)

A-2-3. タイムストレッチ(速い曲への対応)

- 手法: フェーズボコーダー(STFTで時間フレームを分解→フレーム間隔を伸長→再合成)。ピッチは保持したままテンポだけ変更できる
- 用途: リアルタイムでは使えない(未来の音を先読みできないため)。ファイル解析経路でのみ、速いパッセージの時間分解能を確保する目的で使用
- 実装: `soundtouchjs` など既存のJSライブラリで対応可能。自前実装は不要

A-2-4. ポリフォニック経路(Basic Pitch)

- Spotify製のBasic PitchはTensorFlow.js版が公開されており、ブラウザ内で完結できる(サーバー不要、これまでのバックエンドレス方針と矛盾しない)
- 入力:音声ファイル(WAV/MP3) → 出力:MIDIノート列(複数音・和音対応)
- 単純なFFTではなく学習済みモデルによる推定のため、倍音による誤認識(FFTだけでは解決しない問題)をある程度回避できる
- 制約:残響の多い録音、ペダル多用、極端に速く複雑なパッセージでは精度が落ちる。これは現在の技術水準の限界であり、UIで「参考情報である」旨を明記する(D-3の誠実性チェックに準拠)

A-2-5. 正規化アダプター

- YIN経路・Basic Pitch経路のどちらの出力も `{ time: number, pitchClass: 0-11, midiNote: number, confidence: number }` の配列に変換する共通インターフェースを1枚挟む
- 数学モジュール側(Tonnetz、フーリエ解析等)はこの共通フォーマットのみを消費する設計にし、入力経路の違いを一切意識しない

A-3. キー判定(Krumhansl-Schmuckler)

アルゴリズム

1. 検出したピッチクラスの出現時間を12次元ヒストグラムに蓄積(直近N秒の移動窓)

2. Krumhansl-Kesslerの実験由来のプロファイル(長調: [6.35, 2.23, 3.48, 2.33, 4.38, 4.09, 2.52, 5.19, 2.39, 3.66, 2.29, 2.88]、短調も同様の12値)と、入力ヒストグラムのピアソン相関係数を24調(12長調+12短調)すべてについて計算
3. 相関が最大の調を推定結果として表示。2位との差が小さい場合は「確信度低」として両方表示する(断定しない設計は本プロジェクト共通の方針)

A-4. ピッチクラス集合のフーリエ解析(モジュール#5の数学)

理論の中身

- 12音のピッチクラス集合を特性ベクトル $x = (x_0, \dots, x_{11})$ (含まれる=1)として表現
- 離散フーリエ変換: $x_k = \sum_n x_n \cdot e^{(-2\pi i k n / 12)}$ ($k = 0..6$ が意味を持つ)
- 各係数の音楽的意味(これがこのモジュールの見せ場):
 - $|X_5|$ (五度成分): 大きいほど「ダイアトニック的」— 長音階は $|X_5|$ が最大級
 - $|X_6|$ (トライトーン成分): 大きいほど全音音階的
 - $|X_4|$: 大きいほどオクタトニック(減七)的
 - $|X_3|$: 長三度サイクル(増三和音)的
 - $|X_1|$: 音が半音階上で一箇所に固まっている度合い
- 可視化: 各係数を棒グラフ+複素位相を円上の針として表示。ユーザーが12音ボタンをトグルするたびリアルタイム更新

A-5. ユークリッドリズム(Bjorklundのアルゴリズム)

アルゴリズム: $E(k, n) = n$ スロットに k オンセットを最大限均等配置。ユークリッドの互除法と同型の再帰的ペア分配で実装(実装量は50行未満)

- 検証用の既知例: $E(3,8) = \underline{x..x..x.}$ 、 $E(5,8) =$ シンコペーテッドなキンティージョ、 $E(2,5)$ 、 $E(5,16)$ など、実在の伝統リズムと一致することを表示に組み込む(Toussaintの論文の主結果であり、このモジュールの物語になる)
- 円形レイアウトで可視化(n 個の点を円周に配置、オンセットを塗る)+Tone.jsのSequenceで再生

Part B. モジュール別 実装手順(Tier 1)

B-1. 共通基盤(最初に作る、6h)

手順:

1. Next.js (App Router) + TypeScriptでプロジェクト生成
2. ModuleShell.tsx: 全モジュール共通レイアウト
[タイトル / 数式・理論の短い解説 (KaTeXでレンダリング) / 可視化Canvas / コントロール / 出力]
3. toneEngine.ts: 「音を有効化」処理、Oscillator生成ヘルパー、
平均律/純正律の周波数計算関数 (単体テストを書く: 12回の五度=コンマ誤差1.36%など数値で検証可能)
4. MicInput.tsx: マイク許可UI、権限拒否時のフォールバック表示
5. KaTeX導入 (数式表示。「数学的観点」が主役なので、式は画像でなくちゃんと組む)

B-2. モジュール#1 倍音列・純正律vs平均律(5h)

手順:

1. 倍音列ビジュアライザ: 基音 f に対し $1f \sim 16f$ を鍵盤+周波数軸上に表示
各倍音をクリックで単独再生、「ここまで足す」スライダーで音色が変わるのを体感
2. 音程比較パネル: 完全五度/長三度/完全四度について
[純正比で再生] [平均律で再生] [同時に鳴らす] の3ボタン
同時再生時、波形をCanvasにリアルタイム描画し、うなりの周期を視覚でも見せる
3. セント差の表示: $1200 \cdot \log_2(\text{ratio})$ の式とともに、
純正長三度 386.3¢ vs 平均律 400¢ ($=13.7\text{¢}$ のズレ、五度の 2¢ よりずっと大きい) を明示
→ 「五度はほぼ完璧、三度は妥協している」という平均律の本質を数字で示す

B-3. モジュール#3 Tonnetz(8h)

手順:

1. 座標系: 六角格子。右=完全五度 ($+7 \bmod 12$)、右上=長三度 ($+4$)、右下=短三度 ($+3$)
2. Canvas描画: ノード=ピッチクラス、上向き三角形=長三和音、下向き=短三和音
3. インタクション:
 - 三角形クリック→和音再生+ハイライト
 - 隣接三角形へ移動時、共通2音は固定表示・動く1音だけアニメーション
(例: C major→E minor はCがBへ半音移動するだけ、を線の移動で見せる)
 - P/L/R変換ボタン (ネオ・リーマン変換) で隣接移動を体系的に試せる
4. リアルタイム連携: ピッチ検出結果のピッチクラスをTonnetz上で発光させる
直近数秒の検出音で構成される三和音が特定できたら該当三角形をハイライト

B-4. モジュール#5 フーリエ解析(8h)

手順:

1. 12音トグルUI (円形配置のボタン) + プリセット (長音階/旋律的短音階/全音音階/オクタトニック/五音階)
2. DFT計算 (A-4の式) は12点なので毎フレーム再計算でも軽量
3. 可視化: $|X_1| \sim |X_6|$ の棒グラフ + 各係数の位相を小さな円盤上の針で表示
4. 解説パネル: 「 $|X_5|$ が大きい=五度圏上で固まっている=調性的」の対応表
5. リアルタイム連携: マイクからのピッチクラスヒストグラム (直近10秒) を

リアルタイムでDFTにかけ、「あなたの鼻歌のダイアトニック度」を表示
+ Krumhansl-Schmucklerのキー判定結果を併記

B-5. リアルタイムピッチ検出層(8h)

手順:

1. Day1にPoC(A-2参照)でレイテンシ実測 ← 最優先リスク潰し
2. pitchDetector.ts: YIN(Pitchfinder) + メディアンフィルタ + RMSゲート
3. ピッチクラスヒストグラム蓄積(減衰付き: 古い検出は指数減衰で重みを下げる)
4. 各モジュールへの接続はReact Context経由(MicProviderが検出結果をbroadcast)
5. プライバシー表示: 「音声はブラウザ内で処理され、サーバーに送信されません」を明記
(バックエンドレス構成の利点を信頼性の訴求としても使う)

Part C. 追加モジュール候補(将来拡張のアイデア出し)

Part Aの技術基盤がそのまま使えるもの、という条件で:

候補	中身	使う基盤
連分数と平均律の必然性	$\log_2(3/2)$ の連分数展開から12, 19, 31平均律が「良い近似」として自然に出てくる様子を可視化	数式表示+音生成
不協和曲線 (Plomp-Levelt)	2音の周波数差を連続的にスイープし、理論上の不協和度カーブと実際の聞こえを対応させる	音生成+Canvas
スペクトログラム顕微鏡	マイク入力のFFTをリアルタイム表示し、自分の声や楽器の倍音構造を観察	マイク層 +FFT(AnalyserNode 標準機能)
音律の歴史シミュレータ	ピタゴラス音律/中全音律/ヴェルクマイスター/平均律で同じ曲(短いMIDIフレーズ)を聞き比べ	音生成(周波数テーブル差し替え)
インターバルベクトル電卓	任意のピッチクラス集合のインターバルベクトルとZ-relation(異なる集合が同じ音程内容を持つ現象)の探索	集合論ライブラリ(#5と共通)
リズム必要十分デモ	ユークリッドリズムの「均等性」を数値化(オンセット間距離の分散)し、他の配置と比較	#9と共通
1/fゆらぎメロディ生成器	ホワイトノイズ/1/f/ブラウン運動それぞれの統計でメロディを生成し聞き比べ(Voss-Clarkeアルゴリズム)	音生成+乱数
群論ケーリー	T/I群(二面体群D12)のケーリーグラフを描き、和音への作	

グラフ	用をグラフ上の移動として表示	#6と共通
-----	----------------	-------

これらは全て「ブラウザ完結・著作権フリー・数式ベース」という本プロジェクトの制約を満たす。基盤完成後は1個あたり4-6hで追加可能な見込み。

Part D. 品質・検証手順

D-1. 数値検証(単体テスト対象)

- 平均律五度と純正五度のセント差 = 1.955ϕ (理論値と一致するか)
- ピタゴラスのコンマ = $(3/2)^{12} / 2^7 = 531441/524288 \approx 1.01364$
- $E(3,8)=[10010010]$ 、 $E(5,8)=[10110110]$ など既知のユークリッドリズム
- 長音階のDFTで $|X_5|$ が最大になること
- Krumhansl-Schmucklerに「Cメジャースケールの音を均等に含むヒストグラム」を入れてCdurが1位に来ること

D-2. 体感検証(手動テスト)

- 純正律vs平均律の三度の聞き比べで、うなりの差が実際に聞き取れるか(ヘッドフォン/スピーカー両方)
- マイクで「ドレミファソラシド」を歌い、Tonnetz上の発光とキー判定が追従するか
- iOS Safari / Android Chrome / デスクトップの3環境で音とマイクが動くか

D-3. 誠実性チェック(このプロジェクトの信条)

- ピッチ検出の限界(和音・倍音の強い楽器での誤検出)をUI上に明記しているか
- キー判定が僅差の場合に断定表示していないか
- 各モジュールの理論的出典(Euler 1739 / Tymoczko 2006 / Toussaint 2005 / Krumhansl 1990など)を明記しているか

Part E. リスクと対応

リスク	影響	対応
	リアル	

YINのレイテンシ/精度が期待未満	タイム層(差別化の核)が揺らぐ	Day1のPoCで実測。ダメならバッファ調整→AudioWorklet化→それでも無理なら「録音→数秒後に解析」への設計変更をこの順で検討
Basic Pitch(TF.js)がクラシックの速く複雑な曲で精度不足	ファイル解析モードの説得力が下がる	商用製品でも未解決の領域と認識した上で、①タイムストレッチで時間分解能を確保、②精度の限界をUIで明記、③デモ曲は「解析が成功しやすい曲(単旋律主体や中程度のテンポ)」を選んで見せる、の3点で対応。「完璧な採譜」ではなく「数学的構造を覗く入り口」という位置付けを崩さない
iOS Safariの音声制約	モバイル体験の欠落	Tone.start()をタッチ内で呼ぶ設計を共通化。Day2にiOS実機確認
モジュール数の欲張りすぎ	全部が中途半端	Tier1の4+1に厳格に限定。Tier2以降は「基盤の再利用で1個4-6h」が確認できてから着手
数式・理論の説明が難解になりすぎる	対象層が狭まる	各モジュール「音で聞ける具体例→そこから式へ」の順序を統一(式が先、音が後、にしない)